nanoGenRec: A Reproducible Framework for Landing Semantic-ID Generative Recommendation

Ziqing Ye
yeziqing986@gmail.com

June 7, 2026

Abstract

Generative recommendation has recently reframed large-scale retrieval as autoregressive generation over Semantic IDs, with industrial systems reporting gains from model scaling, reinforcement learning, and end-to-end deployment. However, making this paradigm work outside a production paper remains difficult because the useful unit is a full system rather than a single model: item tokenization, behavior sequence construction, next-token training, constrained decoding, full-recall evaluation, experiment governance, and failure diagnosis must all agree. We present nanoGenRec, an open-source framework that lands Semantic-ID generative recommendation as an executable research workflow. The framework implements Semantic ID tokenization, Transformer+MoE next-token recommendation, SID-constrained beam-search evaluation, YAML-based experiment configuration, duplicate-run detection, queue-based long jobs, and durable experiment logs. On anonymized production behavior data, this framework supports 51 recorded experiments and reproduces several useful GR outcomes: a model-scaling curve $L(N) = 2.522 + 2055.1/N^{0.456}$ from 1.7M to 101.1M active parameters; a production behavior-window study showing non-monotonic data scaling under temporal drift; tokenizer sweeps connecting embedding scale, collision, semantic-neighbor retention, and downstream recall; an M-tier 4B-SID full-eval result of R@500=70.4% and R@10=14.2%; and a post-training alignment case where off-policy ECPO collapses to R@500=2.0% while on-policy candidates recover to 67.8%. We position nanoGenRec as a reproducibility framework and landing record for researchers and engineers turning Generative Recommendation from a paper-level recipe into a working system.

1 Introduction

Generative Recommendation (GR) is moving recommender systems from multi-stage retrieve-and-rank pipelines toward autoregressive generation over item tokens. Industrial reports such as OneRec [5], OneRec-V2 [6], OneMall [4], GenRec [7], OneLive [2], and GR4AD [3] show that Semantic-ID generation, model scaling, and preference alignment can be viable at production scale.

The public literature now contains strong system diagrams and deployment evidence, yet there is still a practical gap for researchers who want to turn those ideas into a running system. A working GR system needs more than a model definition. It needs a tokenizer that maps items into discrete semantic tokens, a behavior preprocessing path that produces autoregressive training sequences, a model and training loop, a constrained generation procedure that maps generated tokens back to items, a full-recall evaluation protocol, and experiment infrastructure that can distinguish real improvements from duplicated baselines, invalid feature plumbing, or inline-evaluation artifacts. In practice, the system either works end to end or the individual components are hard to interpret.

We introduce nanoGenRec, an open-source framework and empirical record for landing Semantic-ID-based generative recommendation. The project is built from production-grounded experiments, with private data and deployment-specific details removed. Its purpose is not to claim a new recommendation algorithm. Instead, it shows how the GR stack can be assembled, executed, evaluated, debugged, and iterated in a reproducible workspace. The accompanying experiment lineage records both positive results and failure modes, making the framework useful as a reference implementation and as an operational checklist.

The claim boundary is explicit. nanoGenRec does not claim online production gains, a new tokenization algorithm, or a new preference-optimization objective. Its contribution is a landed framework: the code paths, experiment contracts, evaluation protocol, and logs needed to make Semantic-ID GR run as a real retrieval system and to reproduce a set of useful GR behaviors under that framework.

This report makes four contributions:

1. **A landed GR framework.** We describe an end-to-end workspace covering Semantic ID tokenization, NTP training, SID-constrained beam search, full-recall evaluation, YAML experiment configuration, duplicate-run checks, and queue-based long jobs.
2. **Reproduced GR behaviors.** Within the same framework, we reproduce model scaling, tokenizer-capacity effects, full-recall baselines, and post-training alignment dynamics that match the concerns raised by recent industrial GR systems.
3. **Production-grounded evidence.** We report a seven-point NTP scaling sweep, a behavior-window study under temporal drift, tokenizer and embedding sweeps, full-eval baselines, and an off-policy/on-policy ECPO comparison.
4. **Failure-aware reproducibility.** We preserve experiment logs for successes, negative results, invalid runs, and post-training collapses, including the conditions under which a GR experiment becomes invalid.

2 Background

2.1 Semantic-ID Generative Recommendation

Semantic-ID GR represents each item as a short sequence of discrete tokens. Recommendation then becomes conditional generation: given a user's behavior history, a model predicts the token sequence of the next target item. This formulation allows recommendation to reuse autoregressive modeling, beam search, MoE scaling, and post-training alignment ideas from generative AI.

Recent industrial systems have explored this direction from complementary angles. OneRec introduced an end-to-end generative architecture and reported production deployment with scaling and reinforcement learning evidence [5]. OneRec-V2 improved scalability with a lazy decoder-only design and real-world preference alignment [6]. OneMall unified several e-commerce scenarios with Semantic Tokenization, Transformer-based generation, and RL alignment [4]. GenRec introduced page-wise NTP, token merging, and GRPO-SR for preference-oriented recommendation [7]. OneLive adapted the paradigm to live-streaming with dynamic tokenization and multi-objective alignment [2]. GR4AD studied production advertising with value-aware learning and dynamic beam serving [3].

These papers establish that GR can work in large industrial systems. nanoGenRec addresses a different layer of the problem: the missing reproducible workspace between those system descriptions and a runnable research implementation. It therefore complements prior work by exposing the

operational interfaces, experiment governance, and failure records needed to iterate on GR outside the original production stacks.

2.2 Reproducibility Gap

GR papers typically report architecture and production results, while the operational details needed for reproduction are scattered across preprocessing, training, evaluation, and experiment management. Several details are easy to miss:

- Inline evaluation during training is often a health check, not a comparable full-recall metric.
- Feature ablations can become invalid when preprocessing stores a feature but training or inference does not consume it.
- Tokenizer proxy metrics can disagree unless collision, balance, and behavior-aware semantic-neighbor retention are tracked together.
- RL-style post-training can collapse when rollout candidates are sampled from a distribution that no longer matches the current policy.

nanoGenRec treats these details as first-class experimental artifacts.

3 The nanoGenRec Framework

nanoGenRec is organized around one practical goal: a researcher should be able to take item embeddings and behavior logs, build Semantic IDs, train a generative recommender, decode valid item candidates, run full-recall evaluation, and trace every reported number back to a config and an experiment log. This turns GR from a collection of paper components into a reproducible loop. The loop is also designed to expose failures: invalid feature plumbing, duplicate baselines, stale behavior windows, collapsed tokenizers, and unstable post-training runs are recorded rather than hidden.

3.1 Pipeline

The framework consists of six stages:

1. **Item representation.** Items are embedded with pretrained representation models. The current experiments use Qwen3 embeddings at several scales.
2. **Semantic ID tokenization.** Embeddings are discretized into 3-token item IDs using residual KMeans and FSQ-family variants.
3. **NTP preprocessing.** User behavior logs are converted into autoregressive training shards, with explicit date windows and evaluation targets.
4. **Generative recommendation.** A Transformer+MoE NTP model predicts item SID tokens from user histories.
5. **Full-recall evaluation.** SID-constrained beam search generates candidate item IDs, and Recall@K is computed with $K \in \{10, 50, 100, 500\}$.

Table 1: Framework components and the landing checks they make reproducible.

Component	Landing check	Artifact produced
SID tokenizer	Items map to valid multi-token IDs	SID cache and tokenizer metrics
NTP preprocessing	Behavior logs become training/eval shards	Date-bound shard configs
Transformer+MoE NTP	Histories predict next-item SID tokens	Checkpoints and train metadata
Constrained decoding	Generated tokens map back to real items	Full-recall candidate lists
Evaluation	Metrics use generated item IDs	Recall@K reports
Experiment runner	Runs are reproducible and deduplicated	YAML configs and registry hashes
Experiment logs	Results and failures are auditable	EXP notes and phase READMEs

6. **Experiment governance.** YAML configs, registry hashes, duplicate-run checks, queue-based execution, and phase-level README files preserve experiment lineage.

Table 1 summarizes the framework as a set of landing checks rather than only software modules.

3.2 Experiment Orchestration

New experiments inherit from a shared YAML base config and override only the parameters being tested. Multi-variant experiments are represented as a variant list. A registry hash excludes runtime paths and environment keys, allowing the runner to surface identical or near-identical historical runs before launching a new job. Queue-friendly wrappers support long-running training and evaluation without turning experiment scripts into brittle chains. Each completed run updates a local record of the hypothesis, design, metrics, and interpretation, so the repository accumulates a recoverable experiment lineage rather than only final checkpoints.

3.3 Research-Agent Workflow

The repository includes an inbox/outbox protocol, paper notes, decision records, and status files. This is a lightweight workflow for human-agent collaboration over a research line. The agent reads papers, proposes ideas, estimates runtime, writes configs, checks prior experiments, and records decisions. The contribution here is operational: the research process is made inspectable and recoverable rather than embedded in transient chat logs or shell history.

3.4 Landing Criteria

We treat GR as landed only when four criteria are met. First, generated token sequences must map back to real item IDs through the tokenizer vocabulary. Second, evaluation must use SID-constrained generation over the item space rather than an inline training proxy. Third, experiment changes must be reproducible from configs and traceable to logs. Fourth, failed runs must be diagnosable from recorded metrics and code-path checks. The empirical study below is organized around these criteria: model scaling tests whether the NTP core works, tokenizer sweeps test whether the item representation is usable, full-recall baselines test whether generation retrieves real items, and post-training experiments test whether alignment remains stable after the SFT stage.

Table 2: NTP model tiers used in the empirical study.

Tier	Embed Dim	Layers	Experts	Active Params
S-tier	256	6	8	∼17.5M
M-tier	512	8	8	∼71.6M
L-tier	512	12	16	∼101.1M

4 Experimental Setup

4.1 Data

The empirical study uses anonymized production behavior data. The largest behavior sweep covers up to 7.85M users and 299.0M raw interactions. After sequence truncation, this corresponds to roughly 445M effective SID tokens. Private data and deployment-specific details are not released; the open-source artifact preserves the code, configs, evaluation protocol, and experiment logs.

4.2 Model Tiers

We use three NTP model tiers, summarized in Table 2. Active parameters count only the routed compute used by each token.

4.3 Evaluation Protocol

Full evaluation uses SID-constrained beam search and reports item recall at several cutoffs. Unless otherwise stated, reported R@K values refer to full evaluation rather than inline training evaluation. This distinction matters because inline evaluation uses a limited candidate set and is intended for training health checks.

5 Framework Validation and Empirical Study

nanoGenRec validates the framework by asking whether the complete loop can reproduce behaviors that matter for GR practitioners. The following sections are therefore not independent benchmark claims. They are evidence that the same framework can support model scaling analysis, data-window analysis, tokenizer diagnosis, full-recall retrieval, and post-training alignment debugging.

5.1 NTP Model Scaling

The first validation target is the NTP core. If the framework has correctly connected SID tokenization, sequence construction, training, and evaluation, model scaling should produce a coherent curve rather than isolated wins. We trained seven model sizes from 1.7M to 101.1M active parameters on a fixed 262M-token dataset. Table 3 reports the results.

The fitted loss curve is:

$$L(N) = 2.522 + \frac{2055.1}{N^{0.456}}. \tag{1}$$

The exponent is close to the 0.489 exponent reported by OneRec-V2 [6]. This is an important framework-level signal: the local implementation reproduces a recognizable GR scaling behavior

Table 3: NTP model scaling on a fixed 262M-token dataset.

Config	Active Params	PPL	Loss	R@10	R@500
scale-01	1.7M	235.1	5.460	1.9%	23.6%
scale-02	3.6M	100.4	4.609	3.7%	31.7%
scale-03	5.1M	69.6	4.243	5.4%	45.6%
scale-04	17.5M	28.1	3.334	9.8%	60.5%
scale-05	34.5M	24.0	3.178	11.5%	62.5%
scale-06	71.6M	20.8	3.037	12.6%	66.2%
scale-07	101.1M	19.4	2.965	13.7%	65.8%

Table 4: Behavior-window scaling. The best loss occurs at 14 days, not at the longest window.

Config	Days	Tokens	Users	PPL	R@500
A-7d-S	7	65M	1.02M	30.60	62.1%
B-14d-S	14	130M	1.69M	27.05	58.5%
C-31d-S	31	262M	3.04M	28.05	60.5%
D-62d-S	62	441M	4.86M	30.03	58.6%
E-90d-S	90	553M	6.18M	31.89	56.2%
A-7d-M	7	65M	1.02M	19.31	70.7%
B-14d-M	14	130M	1.69M	18.96	65.8%
C-31d-M	31	262M	3.04M	19.39	65.8%
D-62d-M	62	441M	4.86M	19.80	68.1%

while preserving full-recall metrics. Recall improves substantially through the S-tier and M-tier regime, while gains flatten between 71.6M and 101.1M active parameters under the fixed data budget. Figure 1 shows the fitted curve.

5.2 Behavior-Window Scaling

The second validation target is data plumbing under production time. We fixed the model tier and varied the behavior window. Standard compute-optimal scaling intuition [1] suggests that more data should reduce loss under an i.i.d. assumption. Production recommendation behavior does not satisfy this assumption. Table 4 summarizes the S-tier and M-tier data sweep.

The main observation is non-monotonicity. Extending the window increases the number of users more than the per-user sequence depth, while older behavior can drift away from the current item pool. In this dataset, the exposure window is short enough that old behavior can become stale. Figure 2 visualizes the effect.

5.3 Tokenizer and Embedding Sweep

The third validation target is item representation. Tokenizer quality affects the irreducible loss floor and the search space exposed to beam decoding. We evaluated 14 tokenizer variants over 0.6B and 4B embeddings, uniform 4096/8192 codebooks, MGMR-style hierarchical codebooks, and FSQ hidden sizes. Table 5 shows representative results.

The decisive variable in this sweep is the 8192 codebook setting, which improves L2 balance and snHR. The 4B embedding improves semantic-neighbor retention, but also requires enough FSQ

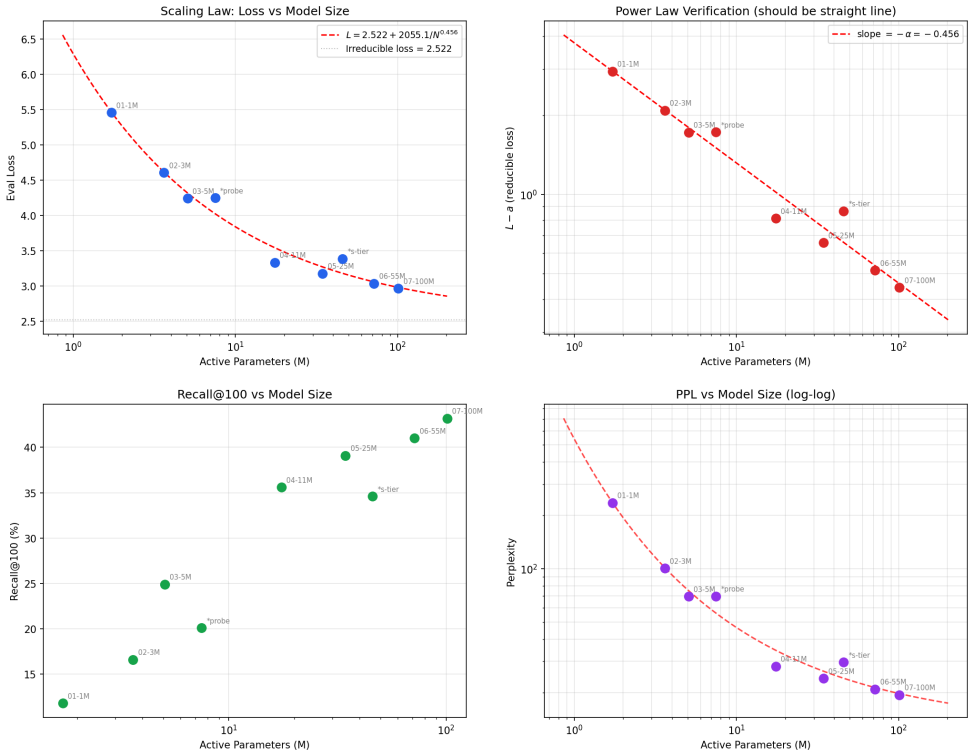

Figure 1: NTP scaling law over active parameters.

capacity to avoid L2 entropy collapse. The current recommended caches are listed in the released experiment logs.

5.4 Full-Recall Baselines

The fourth validation target is the full GR retrieval loop. Table 6 reports current full-eval baselines produced by SID-constrained generation. The strongest observed NTP result is the M-tier model with 4B SID, reaching R@500=70.4% and R@10=14.2%.

5.5 Post-Training Alignment

The final validation target is post-training. We evaluated preference and RL-style alignment methods, including SP-DPO, RF-DPO, GRPO, and ECPO. The most instructive result is the off-policy/on-policy ECPO comparison in Table 7.

The off-policy run generates candidates from a reference model whose distribution drifts away from the current policy. This distorts the importance ratio and causes clipping to dominate. On-policy beam candidates better match the current policy distribution, recovering both PPL and R@500. Figure 3 shows the post-training curves.

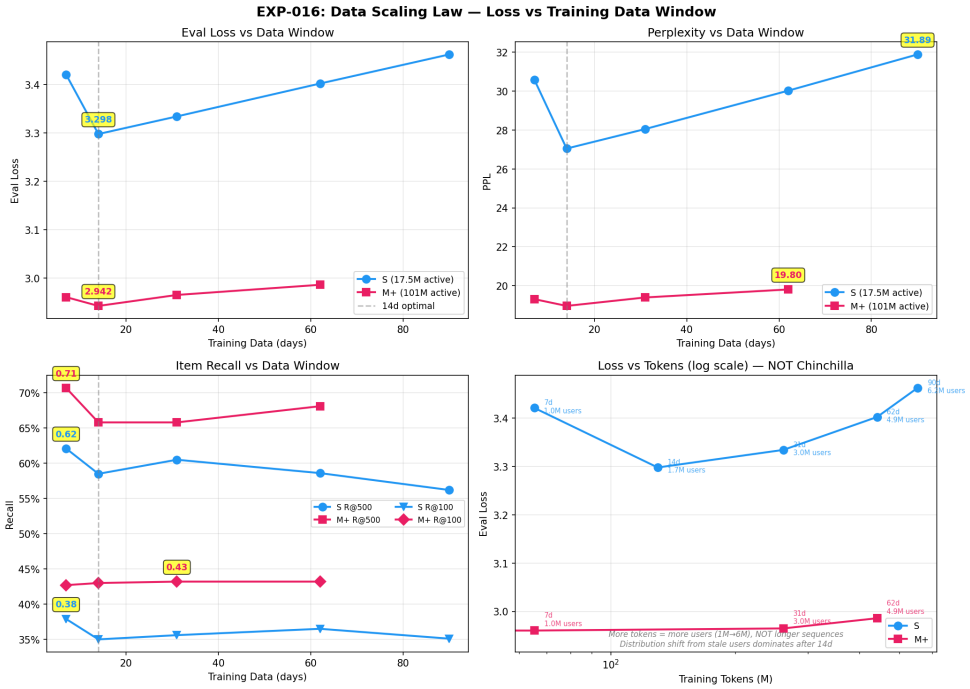

Figure 2: Behavior-window scaling under production temporal drift.

Table 5: Representative tokenizer sweep results. CR is collision rate and snHR is behavior-aware semantic neighbor hit rate.

SID Cache	CR	Gini-d1	Gini-d2	snHR
0.6B nc4096 h64	0.67%	0.3243	0.3546	0.0797
0.6B nc8192 h128	0.42%	0.3914	0.2375	0.0919
4B nc4096 h128	1.60%	0.2877	0.3539	0.1255
4B nc8192 h128	1.28%	0.3192	0.2530	0.1307
4B nc8192x2048 h128	1.41%	0.3191	0.3170	0.1154

6 Discussion

6.1 What the Framework Makes Reproducible

The production data cannot be released. However, the artifact makes the following components inspectable and reusable:

- model and tokenizer implementations;
- preprocessing and evaluation interfaces;
- experiment configs and resolved parameter sets;
- full-recall evaluation protocol;
- experiment logs with hypotheses, designs, results, and post-hoc analysis;
- known invalid runs and bug records.

Table 6: Current full-eval baselines.

Config	R@10	R@500	PPL
S-tier bare	11.4%	61.2%	26.52
S-tier with TO-RoPE ts=0.5	11.8%	63.9%	22.7
M-tier bare, 0.6B SID	14.5%	70.2%	18.54
M-tier, 4B SID	14.2%	70.4%	16.55
L-tier with validated options	12.8%	64.1%	20.7

Table 7: On-policy candidate generation recovers an ECPO collapse.

Config	R@10	R@500	PPL	Clip Rate
Off-policy ECPO	0.7%	2.0%	3791	99%
On-policy ECPO	13.0%	67.8%	14.1	92%
SFT baseline	14.1%	66.2%	16.3	–

This distinction matters. A GR framework can be useful even when a production dataset is not public, provided that the code path, configuration semantics, evaluation protocol, and empirical record are exposed. The main reproducibility target of nanoGenRec is therefore the working loop: how to construct SIDs, train the model, decode valid items, evaluate full recall, organize experiments, and diagnose failures.

6.2 Lessons

Three lessons recur across the experiment lineage. First, landing GR requires full evaluation to be separated from training-time health checks. Second, feature ablations require end-to-end validation across preprocessing, training, and inference; otherwise, experiments can silently compare zeros. Third, post-training alignment in GR is sensitive to candidate distribution, so rollout generation should be treated as part of the algorithm rather than a fixed data-preparation step. These lessons are as much about framework design as model design: most failures arise at component boundaries.

7 Limitations

This report has several limitations. The production behavior data is not released, so external users cannot exactly reproduce the reported production-scale numbers. The framework currently focuses on Semantic-ID autoregressive recommendation rather than all forms of generative retrieval. Online serving, latency, and production integration are not the main focus of the artifact. Some post-training methods are preliminary and should be interpreted as framework-validated engineering evidence rather than final algorithmic conclusions. The most important next step is public benchmark reproduction: adding a fully redistributable dataset path would let external users verify not only the code structure but also the reported empirical trends.

8 Conclusion

We presented nanoGenRec, an open-source framework for landing Semantic-ID generative recommendation. The project contributes an end-to-end GR workflow, full-recall evaluation protocol,

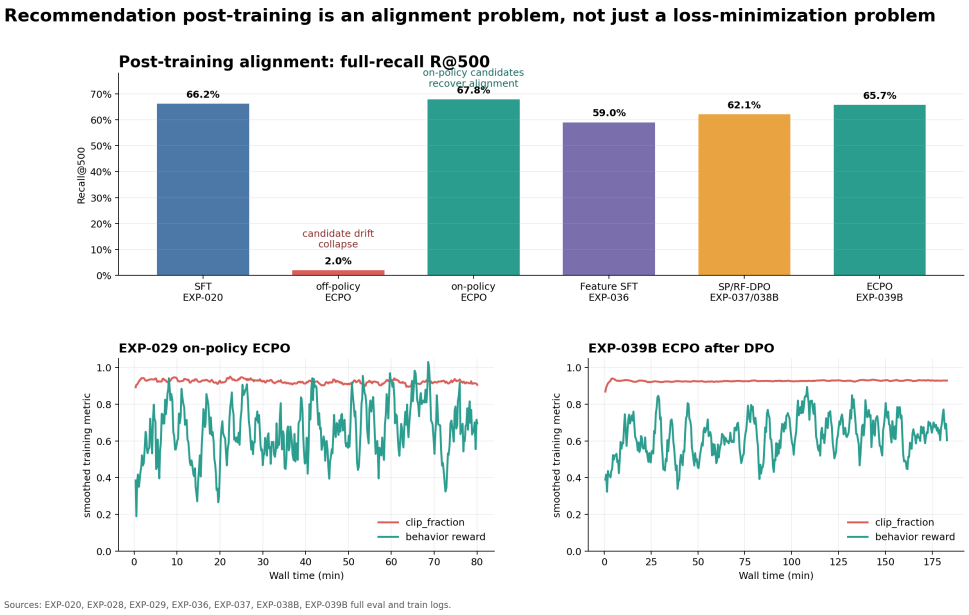

Figure 3: Post-training alignment curves and collapse/recovery behavior.

experiment orchestration system, and 51-experiment lineage. Within this framework, we reproduce several valuable GR outputs: model scaling, non-monotonic behavior-window scaling, tokenizer and embedding effects, full-recall baselines, and post-training alignment collapse/recovery. The central result is not a new standalone algorithm, but a working, inspectable path from Semantic-ID construction to real-item generative retrieval and failure-aware iteration. We hope the artifact helps researchers and practitioners turn Generative Recommendation from a collection of system diagrams into an extensible, production-grounded research workflow.

Artifact. The code and experiment logs are available in the public nanoGenRec repository.[1]

References

[1] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022. URL `https://arxiv.org/abs/2203.15556`.

[2] Shen Wang, Yusheng Huang, Ruochen Yang, et al. Onelive: Dynamically unified generative framework for live-streaming recommendation, 2026. URL `https://arxiv.org/abs/2602.08612`.

[3] Ben Xue, Dan Liu, Lixiang Wang, et al. Generative recommendation for large-scale advertising, 2026. URL `https://arxiv.org/abs/2602.22732`.

[1] `https://github.com/yzq986/nanoGenRec`

[4] Kun Zhang, Jingming Zhang, Wei Cheng, et al. Onemall: One architecture, more scenarios – end-to-end generative recommender family at kuaishou e-commerce, 2026. URL https://arxiv.org/abs/2601.21770.

[5] Guorui Zhou et al. Onerec technical report, 2025. URL https://arxiv.org/abs/2506.13695.

[6] Guorui Zhou et al. Onerec-v2 technical report, 2025. URL https://arxiv.org/abs/2508.20900.

[7] Yanyan Zou, Junbo Qi, Lunsong Huang, Yu Li, Kewei Xu, Jiahao Gao, Binglei Zhao, Xuanhua Yang, Sulong Xu, and Shengjie Li. Genrec: A preference-oriented generative framework for large-scale recommendation, 2026. URL https://arxiv.org/abs/2604.14878.